

LLM Test Generation via Iterative Hybrid Program Analysis

Sijia Gu

The University of British Columbia
Vancouver, Canada
sijiagu@ece.ubc.ca

Noor Nashid

University of British Columbia
Vancouver, Canada
nashid@ece.ubc.ca

Ali Mesbah

The University of British Columbia
Vancouver, Canada
amesbah@ece.ubc.ca

Abstract

Automating unit test generation remains a significant challenge, particularly for complex methods in real-world projects. While Large Language Models (LLMs) have made strides in code generation, they struggle to achieve high branch coverage due to their limited ability to reason about intricate control flow structures. To address this limitation, we introduce PANTA, a technique that emulates the iterative process human developers follow when analyzing code and constructing test cases. PANTA integrates static control flow analysis and dynamic code coverage analysis to systematically guide LLMs in identifying uncovered execution paths and generating better test cases. By incorporating an iterative feedback-driven mechanism, our technique continuously refines test generation based on static and dynamic path coverage insights, ensuring more comprehensive and effective testing. Our empirical evaluation, conducted on classes with high cyclomatic complexity from open-source projects, demonstrates that PANTA achieves 26% higher line coverage and 23% higher branch coverage compared to the state-of-the-art.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging.**

Keywords

Test generation, program analysis, control-flow analysis, large language models

ACM Reference Format:

Sijia Gu, Noor Nashid, and Ali Mesbah. 2026. LLM Test Generation via Iterative Hybrid Program Analysis. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3744916.3764553>

1 Introduction

Automated test generation plays a crucial role in software engineering by enhancing code reliability and maintainability. Large Language Models (LLMs) can replicate human coding styles and generate test cases with meaningful variable names, leveraging patterns learned from extensive training data. This capability has led to the adoption of LLMs for test code generation [1, 4, 7, 15, 22, 24, 28, 35].

Despite their effectiveness in general coding tasks, LLMs—trained on diverse datasets—often struggle with the unique code structures

and logic of specific projects. This limitation is particularly evident in cases involving complex control flow and conditional statements. Recent studies [8, 31, 32, 34] have shown that LLM-generated test cases frequently fail to achieve sufficient branch coverage, as they inadequately capture intricate conditional logic.

To address this limitation, researchers have explored techniques that supplement LLMs with additional program-related information, such as focal method slices [35] and path constraints [28]. These approaches rely on static prompts containing method fragments to generate tests for each independent segment. However, they lack mechanisms to guide the generation process toward areas requiring additional coverage. Furthermore, since LLMs do not always produce correct code, only a subset of the generated tests compile and pass, leaving significant portions of the code untested. Additionally, these methods primarily focus on generating new test classes rather than augmenting existing ones. CoverUp [26] takes a different approach by leveraging code coverage information to highlight uncovered portions of the code. However, our empirical evidence suggests that this approach is insufficient for more complex code, where additional contextual guidance is needed to drive meaningful improvements in branch coverage.

In this paper, we present PANTA (Program ANALysis guided Test Automation), a novel approach that combines dynamic code coverage analysis and static control flow analysis to generate tests for under-tested execution paths. Our approach follows an iterative workflow, implementing a multi-step, feedback-driven fully automated process. PANTA first gathers dynamic code coverage information and static control flow insights. Based on the extracted program analysis information, it identifies and selects execution paths that require additional test coverage for the class, and prompts an LLM to generate test cases that cover those paths. After test generation, PANTA validates the generated tests, adding the passing ones back to the test class while repairing failed tests. The feedback from the test repair process is incorporated into subsequent test generation iterations.

Our work makes the following contributions:

- An iterative feedback-driven technique that integrates dynamic and static program analysis to assist LLMs in generating tests that cover complex control flow paths.
- A novel path selection strategy that first ranks linearly independent execution paths—identified via control flow analysis—using a coverage deficiency score computed from code coverage data. It then selects prioritized paths while incorporating historical selection data to effectively balance exploitation and exploration.
- A fully automated test generation tool, PANTA, that supports both generating new test classes and augmenting existing tests. PANTA is publicly available at [25].



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3764553>

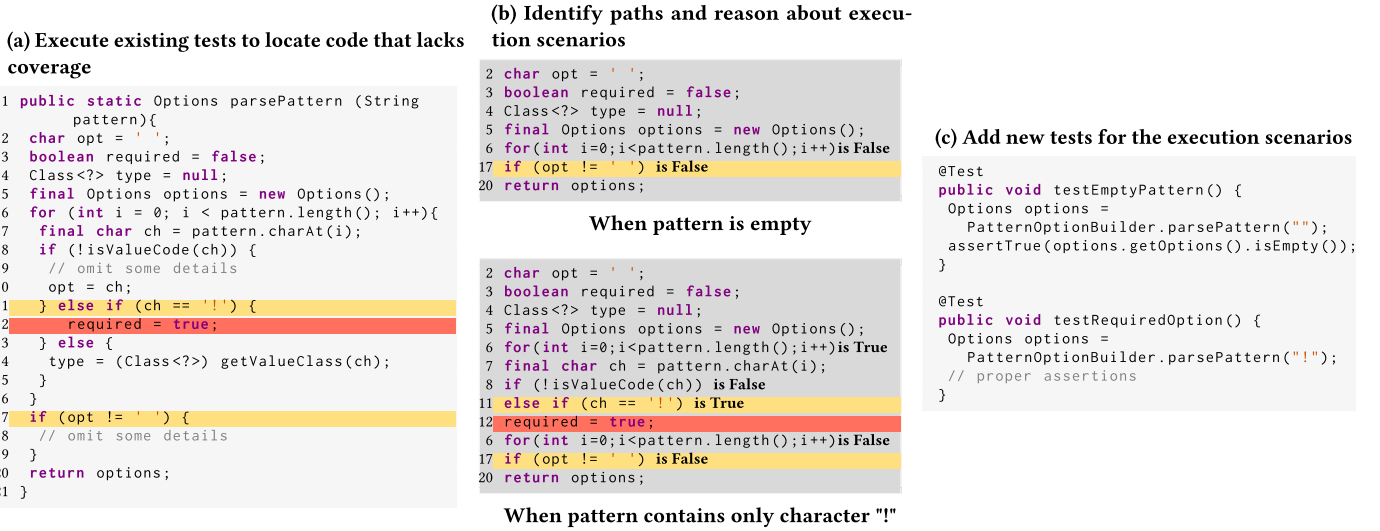


Figure 1: Workflow of a developer to add new test cases

- An empirical evaluation of our approach using real-world open-source programs from the Defects4J [13] benchmark. Our study demonstrates that PANTA achieves 26% higher line coverage and 23% higher branch coverage compared to the state of the art.

2 Motivation

Recent advances in LLMs have dramatically enhanced their code generation capabilities, including the ability to create tests for diverse execution scenarios. However, when dealing with complex methods that contain multiple conditional branches, LLMs often struggle to achieve comprehensive branch coverage.

As a running example, Figure 1a, derived from the Cli project in the Defects4J dataset [13], illustrates this challenge. We prompted Llama 3.3 70b [16], Meta’s latest open-source model, to generate tests for the `parsePattern` method shown in Figure 1a and then measured the resulting coverage. The generated tests, however, failed to cover the true branch of the `else if (ch == '!')` condition at Lines 11–12 and the false branch of the `if (opt != '')` condition at Line 17. We also tried with GPT-4o, which similarly missed the false branch of the condition at Line 17.

Our insight is that emulating a developer’s test creation process can more effectively guide LLMs to generate test cases. Typically, developers begin by checking whether a corresponding test class exists. If it does not, they create a test class skeleton with the necessary imports and gradually add test cases. If it does exist, they run the tests to generate a coverage report that highlights untested statements. They then analyze the control flow—particularly conditional branches and loops—to identify missing execution paths, add new tests to cover these paths, and rerun the tests until adequate coverage is achieved.

For example, Figure 1a shows developers executing the tests and reviewing a JaCoCo [11] coverage report. In the `parsePattern` method, the report indicates that two branches remain uncovered.

By analyzing the control flow, developers determine that reaching the true branch within a `for` loop requires the loop condition to be true while accessing the false branch outside the loop depends on the loop condition being false. Figure 1b illustrates two representative execution paths: one where an empty pattern prevents loop execution and one where a single `'!'` triggers the `else if` branch, which also covers the false branch. New test cases representing these scenarios are shown in Figure 1c.

While developers iteratively refine and expand coverage until achieving the desired adequacy level, manually analyzing the code and creating tests remains a time-intensive task. To address this challenge, our work aims to automate and replicate the iterative workflow that developers follow when generating test cases by combining dynamic and static program analysis to effectively guide LLMs to cover hard-to-cover branches.

3 Approach

Our approach, called PANTA, leverages hybrid program analysis that seamlessly integrates static and dynamic analyses. By iteratively combining static control flow analysis with dynamic code coverage analysis, PANTA enhances LLMs’ ability to determine where and how to generate test cases that improve branch coverage.

Figure 2 provides an overview of this process. It begins by scanning the entire project to identify directories containing source files and their corresponding test files. If a source file lacks an associated test file, a new test file skeleton is created in the designated test directory. For each source file, the hybrid approach first uses static control flow analysis to extract potential execution paths and then employs dynamic analysis—executing the corresponding test file—to collect real-time branch coverage information. The selected paths, derived from a unique path selection strategy discussed in Section 3.2, are incorporated into a test generation prompt that guides the LLM in generating tests.

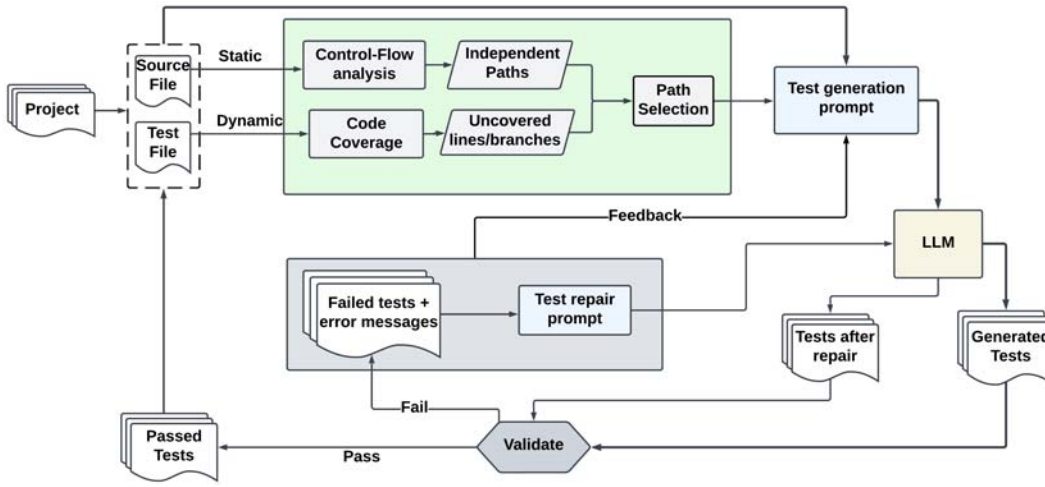


Figure 2: Overview of PANTA.

In addition, the generated tests undergo validation: passing tests are integrated into the test file while failing tests are directed through a test repair procedure. If a test cannot be repaired, it is fed back into the test generation prompt as feedback for the next iteration. This iterative workflow continues until the stopping conditions defined in Section 3.4 are met. Since the process is applied uniformly across all source files and their corresponding test files, we detail the iterative test generation process for a single source file and its associated test file in the following subsections.

3.1 Static Path Approximation

Extracting all execution paths from a source file is often impractical due to the complexity introduced by loop conditions, which can lead to an exponential or unbounded number of paths. To address this, we propose a path approximation strategy through control flow analysis.

We introduce Algorithm 1, a path approximation method that utilizes the control flow graph (CFG) to extract a set of *linearly independent paths*, ensuring that each path introduces at least one new edge. We leverage an existing CFG generation technique [6] to construct the control flow graph for the entire source file, denoted as CFG_f (Line 1). Next, we identify the methods under test (MUTs), excluding helper methods such as private methods, getters, and setters (Line 2). For each MUT, we extract its corresponding method-level control flow graph (CFG_m) from CFG_f (Line 5), forming the basis for our path approximation strategy.

Each CFG_m is a fully directed graph where code blocks correspond to nodes, and control flow determines the directed edges. We compute the cyclomatic complexity (CYC) [20] for each method (Line 6), defined as $\#edges - \#nodes + 2$. For instance, in Figure 3, we illustrate the CFG_m for the `parsePattern` method from Figure 1. We use the line numbers from Figure 1 to represent nodes, where lines 2 to 5 and lines 9 and 10 form basic code blocks that are grouped into single nodes. Circles represent nodes without branches, while branch nodes are marked as pentagons. The directed edges indicate the control flow between nodes. To ensure

Algorithm 1: Path Approximation through CFG

Input: `srcFile`
Output: `methodDict`

```

1   $CFG_f = \text{CFGGraph}(\text{srcFile})$ 
2   $MUTs = \text{identifyMethodUnderTestFrom}(\text{srcFile})$ 
3  /* extract paths for each method under test */
4  foreach  $method$  in  $MUTs$  do
5       $CFG_m = \text{extractCFGGraphForMethod}(CFG_f, method)$ 
6       $CYC_m = \text{computeCyclomaticComplexity}(CFG_m)$ 
7       $paths = \text{ExplorePaths}(CFG_m)$ 
8       $methodDict[method] = (CYC_m, paths)$ 
9  end
10 return  $methodDict$ 
11 /* Explore paths using BFS */
12 Function  $\text{ExplorePaths}(CFG_m)$ :
13      $start, end = \text{getStartEndNodeFrom}(CFG_m)$ 
14      $queue.append([start])$ 
15     while  $queue$  is not empty do
16         for  $i = 0$  to  $\text{length}(queue) - 1$  do
17              $currentPath = queue.pop(0)$ 
18              $lastNode = \text{getLastNodeOf}(currentPath)$ 
19             foreach  $successor$  in  $\text{successors}(lastNode)$  do
20                  $nextPath = \text{copy}(currentPath)$ 
21                  $nextPath.append(successor)$ 
22                 if  $successor$  is not the end node then
23                      $queue.append(nextPath)$ 
24                 else
25                     if  $nextPath$  contains edges not visited then
26                          $methodPaths.append(nextPath)$ 
27                     end
28                     if all edges of  $CFG_m$  has been visited then
29                         return  $methodPaths$ 
30                     end
31                 end
32             end
33         end
34     end

```

a structured representation, we introduce two additional nodes: a start node (1:start) representing the method entry point and a dummy end node (21:end) to unify all execution paths into a single exit. These start and end nodes are depicted as octagons. Excluding

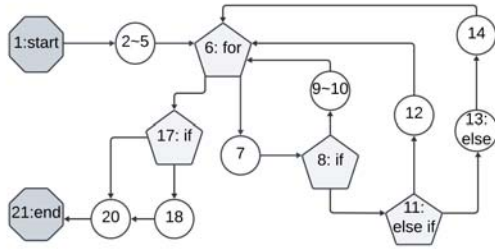


Figure 3: Control flow graph for parsePattern in Figure 1.

these additional nodes, the parsePattern method consists of 12 nodes and 15 edges, with a cyclomatic complexity (CYC) value of 5. We then extract paths using the ExplorePaths in Algorithm 1 (Lines 12–34), which is based on breadth-first search (BFS). The algorithm collects the shortest set of paths, ensuring each includes at least one unique edge. Exploration terminates once all edges have been visited. By employing ExplorePaths, we guarantee that for each method under test, the number of extracted paths equals its CYC.

Using Algorithm 1, we obtain an approximated set of paths for each method under test in a static manner. Next, we use dynamic analysis to identify uncovered execution paths.

3.2 Dynamic Path Selection

We present a path selection strategy in Algorithm 2 that integrates dynamic code coverage data with the static paths extracted from Algorithm 1 to identify and prioritize uncovered paths for each test generation iteration.

Given that multiple test generation iterations may have already occurred, we maintain a *pathHistory* log to track path selection data from previous iterations. Since large language models (LLMs) do not always guarantee the generation of correct tests that fully cover the selected paths in each iteration, a path may be selected multiple times if it remains uncovered. However, if a path has been selected multiple times without being covered, it may indicate that the path itself is invalid or that LLMs struggle to reason about it, making further attempts unproductive. To optimize resource usage, we limit the number of selection attempts per path to *maxSelectedConst*. If a path has been selected *maxSelectedConst* times without being covered, it is excluded from future selections. The data variables, *pathHistory* and *maxSelectedConst*, are used globally throughout the entire test generation process.

In addition to the global data variables, the input to Algorithm 2 consists of the source file, the current test file, and the extracted paths for all methods under test (MUTs) in the source file, denoted as *methodDict*, obtained from Algorithm 1. The output consists of the selected paths for each MUT, which will be incorporated into the prompt for test generation.

After introducing the global, input, and output variables, Algorithm 2 begins by executing the current test suite to collect coverage data, parsing the coverage report to identify missed statements and branches in the source file (Lines 2–3). For each MUT,

Algorithm 2: Path Selection Strategy

Data: pathHistory, maxSelectedConst
Input: srcFile, testFile, methodDict
Output: selectedPaths

```

1 // Run tests and collect coverage information
2 runTestsWithCoverage(testFile)
3 missedLines, missedBranches = parseCoverageReportFor(srcFile)
4 selectedPaths = dict()
5 // Process each method under test
6 foreach method in methodDict do
7   methodPaths = methodDict[method][1]
8   // Filter paths
9   candidatePaths = []
10  foreach path in methodPaths do
11    missedScore = count(missedLines ∩ path) + count(missedBranches
12    ∩ path)
13    if missedScore > 0 then
14      selectedCount = pathHistory[method][path]
15      if selectedCount < maxSelectedConst then
16        candidatePaths.append((path, missedScore,
17        selectedCount))
18    end
19  end
20  if candidatePaths is not empty then
21    // Exploitation: Pick highest missed path
22    highestPath = max(candidatePaths, key=missedScore)
23    pathHistory[method][highestPath] += 1
24    selectedPaths[method].add(highestPath)
25    // Exploration: Pick least selected path
26    leastPath = min(candidatePaths, key=selectedCount)
27    if leastPath ≠ highestPath then
28      pathHistory[method][leastPath] += 1
29      selectedPaths[method].add(leastPath)
30    end
31  end
32 end
33 return selectedPaths

```

we then filter out paths that have been fully covered or have exceeded *maxSelectedConst* selection attempts, retaining the remaining paths as candidates (Lines 6–18). These candidate paths have the potential to increase code coverage, as they contain uncovered statements or branches. To prioritize them, we compute a coverage deficiency score for each candidate path: *missedScore* = num of missed statements in path + num of missed branches in path (Line 11). We count missed statements as uncovered and partially covered lines; for the two paths in Figure 1b, there is one missed statement: line 17 for the above path, and three missed statements: lines 11, 12 and 17 for the below path. To enhance the priority of branch coverage, besides the missed statements, we also increment the count of missed branches, which is 1 (above) and 2 (below) for the two paths respectively. Therefore, the *missedScore* for the two paths are 2 (above) and 5 (below), respectively.

For complex methods with high cyclomatic complexity, the large number of candidate execution paths poses a challenge, making it difficult for LLMs to process efficiently and potentially exceed the available context window. For instance, an untested method with a cyclomatic complexity of 20 could have 20 candidate paths, each representing a distinct execution flow.

To mitigate this, instead of providing all candidate paths in the prompt, we rank them and select two paths per method, based on exploitation and exploration (Lines 19–30). For exploitation, we


```

## Source File
Here is the source file that you will be writing tests against.
{source_file_numbered}

## Test File
Here is the file that contains the existing tests.
{test_file}

## Third-party dependencies for test generation
Please use the following dependencies to generation tests
{test_dependencies}

{%~ if failed_tests_section%}
{{ failed_tests_section }}
{% endif %}

## Methods Under Test
Please generate test for {method_name_1} to cover the path
{selected_path_for_method_1}
...
Please generate test for {method_name_N} to cover the path
{selected_path_for_method_N}

```

Figure 4: Code Structure Aware Prompt Template

choose the path with the highest *missedScore* to maximize code coverage improvement. For exploration, we select the path that has been visited the least in previous iterations, ensuring diversity in test generation. If the two selected paths are the same, we include it only once and update *pathHistory* accordingly. This approach ensures that LLMs have a clearer target and can focus more effectively on generating relevant test cases, without exceeding their context window.

3.3 Program Analysis Guided Prompt Template

After selecting the paths, we construct a test generation prompt by inserting the selected paths into a structured prompt template. Our program analysis-guided prompt template, shown in Figure 4, consists of five sections: source file, test file, test dependencies, failed tests, and methods under test.

The source file section includes the complete source code with additional line numbers to facilitate mapping the selected path lines back to their corresponding locations in the code. The test file section contains the currently existing test cases. The test dependencies section specifies the required packages and their versions in the test scope, such as `junit:4.13.2`, to prevent LLMs from generating tests using incorrect package versions. The failed tests section includes test cases from the last iteration that failed, providing feedback to guide LLMs in refining test generation (subsection 3.4). Finally, the methods under test section includes the selected paths for each MUT, obtained using Algorithm 2. The paths follow the format illustrated in Figure 1b, where each statement is mapped to its corresponding line number in the original source file, and conditional statements are annotated with their branch evaluation (true or false).

3.4 Iterative Test Generation Framework

In the previous subsections, we introduced the algorithms and components used to construct the hybrid program analysis-guided prompt for test generation, focusing on a single iteration. Now, we elaborate on the design of our iterative test generation workflow, which is presented in Algorithm 3 and explain how the various components integrate into the overall PANTA framework.

Algorithm 3: Iterative Test Generation Framework

```

Input: srcFileDir, testFileDir
1 testDeps = extractDependenciesForTestScope()
2 srcFile = readFile(srcFileDir)
3 if testFileDir does not exist then
4   | testFile = initialTestClassSkeleton(testFileDir, testDeps)
5 else
6   | testFile = readFile(testFileDir)
7 end
8 iter, iterNoIncrease = 0, 0
9 pathHistory = dict(), failedTestFeedback = []
10 methodDict = Algorithm 1 (srcFile)
11 maxCYC = getMaxComplexity(methodDict)
12 curCoverage = runTestsWithCoverage(testFile)
13 while iter < maxCYC and iterNoIncrease < maxNoIncreaseLimit and
    curCoverage < 100% do
14   // build prompt for test generation
15   if curCoverage = 0% then
16     | prompt = buildPrompt(srcFile, testFile, testDeps)
17   else
18     | selectedPaths = Algorithm 2 (srcFile, testFile, methodDict,
19       | pathHistory)
20     | prompt = buildPrompt(srcFile, testFile, testDeps, selectedPaths,
21       | failedTestFeedback)
22   end
23   generatedTests = generateTestByPromptLLM(prompt)
24   // Validate generated tests
25   foreach test in generatedTests do
26     stdout = addNewTestAndRun(test, testFile)
27     if test is failed then
28       | errorMsg = parseErrorMessage(stdout)
29       | removeAddedTest(test, testFile)
30       | testsToRepair.add((test, errorMsg))
31     end
32   end
33   failedTestFeedback = repairAndValidate (testsToRepair, numOfAttempts)
34   newCoverage = runTestsWithCoverage(testFile)
35   if newCoverage > curCoverage then
36     | iterNoIncrease = 0
37   else
38     | iterNoIncrease += 1
39   end
40   curCoverage = newCoverage, iter += 1
41 end

```

Test Generation. Algorithm 3 begins by preparing the source file, target test file, and test dependencies (Lines 1–7). For example, if the project uses Maven, we can extract test dependencies using the command `mvn dependency:list`. We then read the source file as context from the directory *srcFileDir*. Our approach supports two scenarios: (1) augmenting existing tests, and (2) generating a new test file from scratch. We first check whether a test file exists at *testFileDir*. If a test file is available, we analyze and augment the existing test case; otherwise, we create a new test file based on the extracted test dependencies, initializing it with the test class declaration and necessary import statements.

We initialize workflow tracking variables (Lines 8–9) to monitor the stopping conditions. Specifically, we use *iter* to count the total number of iterations and *iterNoIncrease* to track the number of consecutive iterations without code coverage improvement. We hypothesize that if several consecutive iterations fail to enhance code coverage, it likely indicates convergence to a fixed value, making further execution unnecessary. Path selection data is stored in *pathHistory*, which is referenced by Algorithm 2 for selecting paths.

Additionally, we maintain *failedTestFeedback* to store failed tests from the previous iteration, providing feedback to refine the current test generation process. Next, we perform program analysis to extract paths for MUTs using Algorithm 1 and obtain the initial code coverage information (Lines 10–12). We calculate the maximum cyclomatic complexity among all MUTs in the source file, denoted as *maxCYC* (Line 11). Since the number of paths for any MUT is at most *maxCYC*, we use this value to approximate the complexity of the source file and set it as the maximum number of iterations for test generation. This ensures that the iterative workflow runs for at most *maxCYC* iterations. Along with this iteration limit, the process terminates under three conditions (Line 13): (1) if the number of iterations reaches the limit *maxCYC*, (2) if the code coverage reaches 100%, or (3) if there are *maxNoIncreaseLimit* consecutive iterations without coverage improvement. These conditions help optimize resource usage and prevent unnecessary test generation attempts.

For each iteration, we determine the test generation strategy based on the current code coverage. If the coverage is 0%, indicating that no existing tests are available, we prompt LLMs to generate tests using only the source file, test file, and test dependencies, without providing the selected paths. This approach allows greater flexibility for LLMs to generate tests covering different parts of the source file (Lines 15–17). If the coverage is greater than 0%, we guide LLMs using our program analysis-guided prompt, which includes the selected paths to focus test generation on uncovered parts of the code (Lines 18–20).

Test Validation. After the LLM completes test generation, the generated tests are directed to the validation step (Lines 23–30). The tests are executed one by one. Passed tests are added to the test file, while failed tests, along with their error messages, are collected for the test repair process (Line 31). We prompt the LLM with the failed tests and their corresponding error messages, allowing it to attempt repairs for up to *numOfAttempt* times per failed test. We observed that if a test initially fails due to a compilation error, fixing the compilation error may reveal additional compilation errors or runtime errors. Therefore, a single repair attempt is often insufficient. To address this, we set the default value of *numOfAttempt* to 3. The repair process stops when either all failed tests are successfully fixed or the maximum number of repair attempts is reached. After the repair attempts, if any tests still fail, we store them along with their error messages in *failedTestFeedback*, which serves as feedback for the next round of test generation. Finally, we execute the current test file to obtain the updated coverage information and update the *iter* and *iterNoIncrease* variables (Lines 32–39). If none of the stop conditions have been met, the process continues with the next iteration.

4 Evaluation

We assess PANTA to answer the following three research questions:

- **RQ1:** How does the quality of tests generated by PANTA compare to state-of-the-art test generation techniques?
- **RQ2:** What is the individual contribution of each component of PANTA to the overall quality of the generated tests?
- **RQ3:** How do different LLMs affect the performance of PANTA in test generation?

Table 1: Included subjects from Defects4J

Identifier	Project	#Class	#MUTs	CYC _{max}	CYC _{med}
Cli	Cli-40f	2	31	11	1
Codec	Codec-18f	7	78	31	1
Collections	Collections-28f	5	218	32	2
Compress	Compress-47f	9	65	22	3
Csv	Csv-16f	3	72	25	1
Gson	Gson-16f	4	75	37	2
JCore	JacksonCore-26f	9	161	30	4
JDataBind	JacksonDataBind-112f	9	371	22	2
JXml	JacksonXml-5f	4	118	29	2
Jsoup	Jsoup-93f	8	171	22	1
JxPath	JxPath-22f	12	115	32	4
Lang	Lang-4f	17	586	30	2
Math	Math-2f	30	452	31	2
Time	Time-13f	11	458	30	2
	Total/Ave	130	2971	27	2

4.1 Experimental Setup

Subject Programs. We evaluate PANTA using real-world subjects from Defects4J [13]. Specifically, we selected the latest fixed version of each subject program from Defects4J v2.0.1. To maintain consistency, we exclude three outdated projects—Chart, Mockito, and Closure—as they do not use Maven and rely on deprecated dependencies. The remaining 14 subjects are compatible with Java 8 or higher, Maven (v3.6.3), and JUnit (v4.13.2). They are further integrated with JaCoCo (v0.8.11) [11] for code coverage measurement and Pitest (v1.17.0) [5] for mutation score evaluation.

To evaluate coverage on complex code, we did not generate tests for all classes; instead, we selected non-abstract, public classes that contain at least one method with cyclomatic complexity greater than 10 [20, 30]. Our process began by automatically selecting all public, non-abstract classes from the 14 Defects4J projects that include at least one non-private method exceeding this threshold. We then manually excluded two types of outliers: (1) classes with extreme complexity (e.g., cyclomatic complexity > 40) caused by large switch-statement patterns, and (2) classes with minimal explicit control flow due to heavy reliance on inheritance. These exclusions ensured that our evaluation remained focused on complex yet analyzable control-flow structures. Table 1 presents the selected subjects. The column Project denotes the class name along with its version number from Defects4J. The column #Class represents the number of classes for which we generate tests, while #MUTs counts the total number of methods under test within the selected classes, for each project. In total, we evaluate 130 classes containing 2,971 methods under test (MUTs). Additionally, we report the maximum and median cyclomatic complexity values of the methods under test in the columns CYC_{max}, CYC_{med}, respectively. More details about individual classes are publicly available in our repository [25].

Evaluation Metrics. We adopt commonly used quality metrics in test generation literature [7, 28, 35], including *line coverage*, *branch coverage*, and *pass rate* for RQ1 and RQ2. The pass rate is defined as the percentage of generated tests that successfully pass when added to the test suite. Since we evaluate at the class level rather than the method level, we measure line and branch coverage per class and report the average coverage across all targeted classes for each

Table 2: Comparing PANTA with SymPrompt

Identifier	#Class	line coverage (%)		branch coverage (%)		pass rate (%)		mutation score (%)		HCC(#)	
		PANTA	SymPrompt	PANTA	SymPrompt	PANTA	SymPrompt	PANTA	SymPrompt	PANTA	SymPrompt
Cli	2	83.56	42.93	70.02	34.02	85.21	29.31	39.31	33.53	1	0
Codec	7	81.83	47.67	77.05	41.77	72.82	29.04	26.10	19.14	5	1
Collections	5	72.66	61.77	68.53	64.48	40.01	49.83	63.43	55.56	2	3
Compress	9	56.95	21.90	46.19	17.31	43.40	20.29	25.16	9.09	1	0
Csv	3	76.99	46.47	56.74	35.48	69.88	34.45	69.45	48.45	0	0
Gson	4	77.07	40.81	60.95	32.57	72.96	10.44	46.17	23.74	1	0
JCore	9	68.43	36.21	60.88	33.58	61.74	36.32	29.79	12.13	3	1
JDatabind	9	56.77	35.64	47.10	29.93	28.99	23.11	35.34	39.85	1	0
JXml	4	67.81	26.32	62.99	22.26	64.18	26.31	41.10	7.82	1	1
Jsoup	8	82.99	36.99	61.98	25.62	68.93	22.92	56.69	32.89	2	0
JxPath	12	56.80	24.40	49.05	22.48	50.97	14.08	45.97	20.73	4	1
Lang	17	71.18	57.71	64.13	52.18	72.27	51.39	39.01	53.31	5	2
Math	30	70.94	50.45	62.81	44.75	41.38	35.57	43.05	36.68	13	6
Time	11	80.57	58.66	70.34	47.41	71.86	45.05	52.45	45.33	6	1
Total/Ave	130	70.18	43.91	60.83	38.17	55.92	32.83	43.79	31.30	45	16

project. For RQ1 and RQ3, in addition to these standard metrics, we use two additional metrics: *High Coverage Count (HCC)* and *mutation score*. Based on industry standards [3, 12], typical code coverage targets range from 70% to 80%, e.g., Salesforce sets a 75% threshold for line coverage [29]. However, since line coverage is generally easier to achieve than branch coverage, we focus primarily on branch coverage. Therefore, we report the number of classes that achieve at least 75% branch coverage as a high coverage count, denoted as HCC. Additionally, we evaluate *mutation score* to assess test effectiveness, as coverage alone does not ensure fault detection if test assertions are non-existent or weak [39].

LLM Selection. We select the latest open-source model, Meta Llama 3.3 70B [16], as our primary model for evaluation. For RQ3, to assess the impact of different models, we evaluate PANTA on three additional models: one open-source model, Mistral Large 2 [21] from Mistral AI, and two proprietary cost-efficient models—GPT-4o Mini from OpenAI [23] and Claude 3.5 Haiku from Anthropic [2]. All LLMs evaluated support a context window of 128k tokens. The model parameters are held constant across all four LLMs used in our evaluation, with `max_tokens` set to 4096 for output generation and temperature set to 0.2.

PANTA supports both augmenting existing test classes and generating new ones. To ensure a fair comparison, we evaluate PANTA in the scenario where tests are generated for a new class without any existing test cases across all research questions. Given the low temperature, the generation process is largely deterministic, with only minor fluctuations. As a result, we perform a single run per class and report the average results across all classes for each project.

4.2 RQ1: Generated Test Quality Comparison

Search-based tools such as EvoSuite [9] have already been extensively studied in comparison with LLM-based test generation [4, 7, 27, 28, 35]. Prior results demonstrate that LLM-based tools have advanced beyond search-based approaches by achieving higher average line and branch coverage, although EvoSuite remains competitive on certain projects. Moreover, search-based generated tests

are often difficult to interpret and lack meaningful assertions compared to those produced by LLM-based methods [7, 27]. Because our contribution lies in advancing LLM-based test generation, we therefore focus our comparisons on other LLM-based tools. The latest test generation techniques relevant to PANTA include HITS [35] and SymPrompt [28]. HITS decomposes focal methods into slices and generates tests slice by slice, while SymPrompt extracts path constraints as a list of branch conditions for each method under test and generates tests path by path. Neither approach leverages dynamic code coverage data and both operate statically without guidance on where additional tests are needed. Consequently, HITS and SymPrompt are limited to generating new test classes rather than augmenting existing ones. To differentiate PANTA with them, PANTA adopts an iterative workflow, guiding the LLM to generate tests for selected paths based on both dynamic and static program analysis information. Our approach works for both scenarios to augment existing tests and add new test classes.

However, neither HITS nor SymPrompt has publicly available implementations. HITS relies on an LLM to generate slices instead of traditional program analysis, and the paper [35] provides limited implementation details, making replication challenging. In contrast, SymPrompt provides sufficient details to replicate its approach. Therefore, to compare it with PANTA, we implemented SymPrompt’s technique following the information that is available in their paper [28].

We present the results in Table 2. We highlight the best values in bold to enhance visualization in the result tables. In terms of line and branch coverage, PANTA consistently outperforms SymPrompt across all projects.

On average, PANTA achieves 26.3% higher line coverage and 22.7% higher branch coverage than SymPrompt. Regarding high coverage count (HCC), 45 out of 130 classes achieve branch coverage above 75%, compared to only 16 for SymPrompt. The pass rate of SymPrompt is, on average, 23.1% lower than PANTA. Beyond the absence of a test repair phase, this can be attributed to its limited prompt context. Specifically, SymPrompt provides only

Table 3: Ablation Study

Identifier	#Class	line coverage (%)				branch coverage (%)				pass rate (%)			
		baseline	PANTA_basic	PANTA_cov	PANTA	baseline	PANTA_basic	PANTA_cov	PANTA	baseline	PANTA_basic	PANTA_cov	PANTA
Cli	2	24.29	78.72	85.12	83.56	13.12	63.99	71.93	70.02	28.57	77.64	66.25	85.21
Codec	7	39.06	77.91	65.55	81.83	32.24	69.46	57.86	77.05	41.51	62.67	59.85	72.82
Collections	5	30.30	66.85	64.97	72.66	29.90	63.74	66.32	68.53	60.80	76.38	75.04	40.01
Compress	9	17.76	47.40	47.59	56.95	14.02	39.06	39.86	46.19	19.88	49.90	40.54	43.40
Csv	3	12.11	66.18	59.34	76.99	6.00	51.25	44.49	56.74	28.70	61.11	61.02	69.88
Gson	4	28.00	62.00	75.46	77.07	17.00	52.30	55.85	60.95	52.92	64.47	64.46	72.96
JCore	9	25.47	40.70	42.37	68.43	15.89	33.17	35.02	60.88	42.60	42.65	45.32	61.74
JDatabind	9	17.90	40.49	38.43	56.77	12.17	30.95	29.74	47.10	37.10	47.37	43.23	28.99
JXml	4	12.26	53.70	38.08	67.81	4.04	40.89	30.37	62.99	25.22	60.71	37.50	64.18
Jsoup	8	42.35	65.80	77.62	82.99	27.99	49.85	61.02	61.98	55.90	52.41	65.19	68.93
JXPath	12	30.77	53.28	52.35	56.80	23.89	45.41	45.42	49.05	42.08	59.60	47.51	50.97
Lang	17	34.47	60.90	58.72	71.18	29.85	53.74	51.34	64.13	65.31	67.29	64.92	72.27
Math	30	37.47	66.73	62.35	70.94	29.39	56.14	55.58	62.81	61.84	55.69	54.96	41.38
Time	11	55.47	71.42	68.84	80.57	40.96	59.40	57.84	70.34	86.71	65.77	66.53	71.86
CYC _{max} ≤ 20	104	34.01	61.56	60.07	70.37	26.29	51.80	51.86	61.09	51.42	58.21	55.60	56.10
CYC _{max} > 20	26	27.38	55.49	52.68	69.42	19.17	46.00	42.93	59.81	56.55	59.15	56.66	55.16
Total/Ave	130	32.69	60.34	58.59	70.18	24.87	50.64	50.08	60.83	52.45	58.40	55.81	55.92

branch conditions rather than the full execution path, making it more challenging for the LLM to infer correct test inputs. In terms of mutation score, PANTA achieves a higher average score (43.8%) than SymPrompt (31.3%), aligning with its superior code coverage results.

There are a few scenarios in which SymPrompt performs better. In the Collections project, SymPrompt achieves a higher pass rate than PANTA, while in JDatabind and Lang, its mutation score exceeds that of PANTA. Upon analysis, these scenarios occur in large utility classes containing large number of methods. For example, MapUtils in Collections has 71 methods. Since PANTA generates tests at the class level, it selects paths for each MUT that have not been fully covered in each iteration. For large classes, this results in long prompts, increasing the difficulty for the LLM to generate complete tests. In contrast, SymPrompt operates at the method level, allowing it to focus on smaller contexts, making it more effective in these specific cases.

Overall, across all evaluation metrics—line coverage, branch coverage, pass rate, HCC, and mutation score—PANTA consistently outperforms SymPrompt.

4.3 RQ2: Ablation study

To assess the contributions of each component of PANTA, we design different variants that provide the LLM with varying levels of information for test generation. We evaluate PANTA in a scenario where tests are generated for a new class without any existing test cases. As shown in Algorithm 3, the essential inputs include the source file (srcFile), test dependencies (testDeps), and a new test file (testFile), initialized with the test class declaration and necessary import statements.

To establish a baseline, we prompt the LLM with only this basic information for a single test generation pass, without leveraging multiple iterations, test repair, or feedback, which are key features of our iterative framework. To assess the impact of our iterative

framework, we introduce PANTA_basic, a variant that generates prompts consistently across all iterations using only the basic information, without incorporating any program analysis data. Additionally, we evaluate PANTA_cov to determine whether code coverage information alone can enhance test coverage. The PANTA_cov variant extends the basic information by incorporating code coverage data, providing the LLM with the locations of uncovered lines and branches in the current class as line numbers mapped to the original source. However, it does not include path selection details on how to cover these uncovered elements.

The results comparing the baseline, PANTA_basic, PANTA_cov, and PANTA are presented in Table 3. In addition to reporting the average line/branch coverage and pass rate for each project, we classify the 130 evaluated classes based on their maximum cyclomatic complexity (CYC_{max}), using two groups: CYC_{max} ≤ 20 and CYC_{max} > 20. A cyclomatic complexity greater than 20 is considered an indicator of highly complex programs [30].

For line and branch coverage, we observe a substantial improvement when comparing PANTA_basic to the baseline, with line coverage increasing by 27.65% and branch coverage by 25.77%. This demonstrates that our iterative framework significantly enhances overall code coverage. While PANTA_cov achieves similar coverage, its average coverage is slightly lower than that of PANTA_basic. A Paired T-test [19] confirms that the difference between them is not statistically significant across all subjects. Notably, for the Cli project, which has lower complexity (CYC_{max} = 11), PANTA_cov achieves the highest line and branch coverage. This suggests that for less complex subjects, code coverage information alone can be beneficial, but for more complex projects, it is insufficient to drive meaningful improvements in coverage.

By integrating code coverage-driven path selection (Algorithm 2), PANTA further improves line and branch coverage by approximately 10% compared to the other two PANTA variants. When analyzing coverage across complexity categories, we observe a notable drop

Table 4: Comparison among different models

Identifier	line coverage (%)				branch coverage (%)				pass rate (%)				mutation score (%)				HCC(#)			
	Llama	Mistral	GPT	Claude	Llama	Mistral	GPT	Claude	Llama	Mistral	GPT	Claude	Llama	Mistral	GPT	Claude	Llama	Mistral	GPT	Claude
Cli	83.56	58.04	75.79	97.61	70.02	39.38	54.41	82.33	85.21	33.52	66.73	86.30	39.31	33.53	47.98	74.57	1	0	0	2
Codec	81.83	73.20	55.24	86.20	77.05	68.00	49.45	80.82	72.82	41.40	43.72	69.48	26.10	36.37	25.35	48.14	5	4	1	5
Collections	72.66	78.64	60.73	61.10	68.53	77.61	58.13	62.95	40.01	51.59	53.64	82.73	63.43	70.89	62.87	62.87	2	4	2	3
Compress	56.95	31.14	37.52	74.83	46.19	27.44	29.13	70.08	43.40	16.99	23.67	61.61	25.16	11.82	13.98	30.05	1	0	0	4
Csv	76.99	69.17	46.39	62.08	56.74	58.61	28.77	46.89	69.88	33.16	24.23	55.13	69.45	67.78	52.98	68.02	0	0	0	0
Gson	77.07	48.32	73.32	79.33	60.95	37.67	56.48	61.24	72.96	20.02	34.58	62.03	46.17	25.29	47.99	53.05	1	1	0	1
JCore	68.43	45.73	44.75	65.90	60.88	40.25	39.27	61.04	61.74	31.65	35.72	69.16	29.79	13.32	13.61	25.48	3	2	1	4
JDatabind	56.77	45.96	52.50	52.45	47.10	42.99	46.70	49.70	28.99	23.22	49.83	62.81	35.34	37.14	35.83	38.12	1	2	1	2
JXml	67.81	41.88	34.59	34.02	62.99	38.66	32.41	38.17	64.18	31.23	31.88	25.11	41.10	17.64	13.04	15.95	1	1	1	1
Jsoup	82.99	60.00	68.25	80.22	61.98	45.75	52.87	65.62	68.93	38.82	48.68	75.29	56.69	42.70	45.45	59.65	2	1	1	3
JxPath	56.80	50.85	32.66	65.85	49.05	42.17	29.20	57.78	50.97	29.07	29.04	52.02	45.97	43.38	29.90	50.48	4	1	1	4
Lang	71.18	65.30	65.00	75.61	64.13	58.85	58.64	71.57	72.27	39.63	56.24	82.44	39.01	32.43	33.05	39.42	5	5	4	9
Math	70.94	56.04	55.02	81.27	62.81	49.62	49.65	72.87	41.38	40.77	51.01	73.03	43.05	28.30	30.56	44.86	13	8	8	16
Time	80.57	70.99	64.26	79.81	70.34	63.61	54.25	74.15	71.86	45.37	47.05	64.81	52.45	51.85	35.46	60.24	6	5	2	8
Total/Ave	70.18	56.60	54.05	73.20	60.83	49.85	46.77	66.52	55.92	35.48	44.39	68.16	43.79	36.60	34.86	47.92	45	34	22	62

of 6% to 9% for highly complex subjects ($CYC_{max} > 20$) in the baseline, PANTA_basic, and PANTA_cov variants. In contrast, PANTA maintains nearly consistent coverage across both complexity levels, with only a 1 percentage point difference. These findings demonstrate that our iterative framework, combined with hybrid program analysis leveraging both static and dynamic information, effectively enhances code coverage, particularly for complex subjects.

The pass rate is primarily determined by the LLM’s ability to generate correct code. Since all variants use the same LLM (Llama 3.3 70B), the pass rate remains relatively stable. PANTA_basic achieves a 6% higher pass rate than the baseline, attributed to the test repair phase incorporated in our framework. However, after integrating program analysis information into the prompt, the overall pass rate is slightly lower than PANTA_basic, with only a 3.5% improvement over the baseline. This trend holds for both PANTA_cov and PANTA. The reason is that prompts in PANTA_cov and PANTA guide the LLM to target more complex and hard-to-cover execution paths, making it more challenging to generate correct test cases. In contrast, PANTA_basic offers greater flexibility, allowing the LLM to decide where to add tests. As a result, the LLM tends to avoid complex paths, leading to a higher pass rate by focusing on easier-to-cover cases.

4.4 RQ3: Impact of different models

In addition to Llama 3.3 70B, we evaluate PANTA on three other models, as introduced in Section 4.1. Table 4 presents the results across four models: Llama 3.3 70B (Llama), Mistral Large 2 (Mistral), GPT-4o Mini (GPT), and Claude 3.5 Haiku (Claude) for each evaluation metric.

For code coverage metrics, Claude performs the best, achieving 73.2% line coverage, 66.5% branch coverage, and 62 out of 130 classes surpassing 75% branch coverage. Llama follows, with 70.2% line coverage, 60.8% branch coverage, and 45 classes exceeding the 75% threshold. GPT performs the worst in achieving high coverage. Thus, the ranking for code coverage is Claude > Llama > Mistral > GPT. The same ranking applies to mutation score, as Claude also

achieves the highest score, followed by Llama, Mistral, and GPT. For pass rate, which reflects the model’s ability to generate correct code, Claude again performs the best (68.2%), with Llama as the second highest (55.9%). However, unlike the coverage metrics, GPT achieves a higher pass rate than Mistral, resulting in a ranking of Claude > Llama > GPT > Mistral for pass rate.

Overall, Claude is the best-performing model, but as a proprietary model, it has the highest API pricing among the four models. For a more cost-effective alternative, Llama achieves results comparable to Claude while being open-source, making it a more budget-friendly option.

5 Discussion

Limitations. Regarding the sufficiency of source code context, PANTA analyzes one class at a time, capturing constructors and internal methods but omitting external dependencies across classes. This design is less effective for classes that rely heavily on inheritance, where key behaviors are defined through method overrides in parent classes. In such cases, the absence of explicit method implementations reduces the effectiveness of control flow analysis. However, since our study examines whether hybrid program analysis improves coverage for complex methods and all evaluated prompting strategies operate in a class-level context, the RQ2 ablation study - conducted under identical conditions - demonstrates a clear improvement with hybrid analysis, validating its effectiveness even within this limited scope. In the future, we plan to extend our approach to incorporate cross-class context to further address this limitation. Regarding test repair, a common limitation across different LLMs is their inability to effectively fix runtime errors, such as *AssertionError* caused by failed test assertions. In contrast, LLMs perform better in resolving compilation errors. This indicates that LLMs lack the capability to fully understand project-specific execution logic, as they struggle to infer runtime behaviors and expected outcomes. Regarding test effectiveness, we observe that the average mutation score across different LLMs consistently falls below 50%, indicating that less than half of the introduced mutants are detected.

This suggests that LLM-generated tests still have significant room for improvement in enhancing fault detection capability.

Application. The tests generated by PANTA operate under the assumption that they are created within a regression testing setting, where the correctness of the underlying focal method implementation is presumed. As PANTA supports both augmenting existing test classes and generating new ones, it can be integrated with any test generation tool to further improve code coverage in their generated test suites. Our evaluation focuses on Java SE projects from the Defects4J benchmark, which are compatible with Java 8 and newer versions. We select this benchmark because it includes projects with rich control-flow complexity, aligning with our goal of improving coverage for hard-to-cover branches. Consequently, PANTA is most applicable to Java SE projects with similar structural complexity. Extending the approach to Java EE applications, which pose additional challenges such as dependency injection and container management, remains an important direction for future work.

Runtime. In our evaluation of 2,971 methods across 130 classes, the average runtime for PANTA to generate a passing test suite is 2.3 minutes per method, or approximately 53 minutes per class. Runtime varies with class complexity, method count, and project characteristics. For example, in Cli-40f, classes contain an average of 15 methods with a maximum cyclomatic complexity of 11, resulting in 0.8 minutes per method (12 minutes per class). In contrast, in Collections-28f, classes average 44 methods with cyclomatic complexity up to 32, leading to 9.4 minutes per method (111 minutes per class). Large utility classes with extensive iteration and map usage typically lead to longer generation times. The longer runtimes reflect the challenging nature of the selected classes, which include hard-to-cover branches and high structural complexity. Improving runtime efficiency remains an important avenue for future work.

6 Threats To Validity

Internal Validity. The effectiveness of PANTA depends on the capabilities of the LLM used, and results may vary if more advanced models are introduced in the future. To mitigate this, we evaluate PANTA using four recently released LLMs, including two open-source and two proprietary models. Another potential issue is the relationship between code coverage and test effectiveness, as high coverage does not necessarily imply meaningful assertions. To address this, we include mutation score in our evaluation to assess fault-detection capability alongside traditional coverage metrics. Additionally, PANTA integrates Comex [6] to generate control flow graphs for path extraction. Any bugs or limitations in Comex could potentially influence our results. To mitigate this risk, we verify the extracted paths during the implementation phase to ensure their correctness before they are used for test generation. Another concern is data contamination, i.e., the possibility that LLMs may have encountered parts of the Defects4J codebase during pretraining. We recognize this as a broader challenge in LLM-based software engineering research. To mitigate its effect and ensure fair comparisons, all evaluated prompting strategies use the same set of projects and the same underlying LLM. Our evaluation therefore reports relative improvements between prompting strategies rather than absolute performance.

External Validity. Our current implementation and evaluation focus on Java, which may impact the generalizability of our findings. However, both LLM-based test generation and program analysis techniques used in PANTA are language-agnostic. The integration of dynamic code coverage analysis and static control flow analysis can be extended to support multiple programming languages, provided that equivalent tooling is available.

Reproducibility. To ensure reproducibility, we have made PANTA's implementation and dataset publicly available [25]. Additionally, we provide detailed instructions for replicating our experimental results.

7 Related Work

Automated unit test generation has been a longstanding research topic, with approaches historically falling into two main categories: search-based and learning-based. Popular search-based approaches, such as EvoSuite [9, 10] for Java and Pynguin [17, 18] for Python, utilize evolutionary algorithms to generate high-coverage unit tests. However, these approaches often suffer from generating test cases that are difficult for developers to understand and may lack meaningful assertions [7, 27]. On the other hand, learning-based approaches leverage deep learning techniques to improve test case quality. For instance, ATLAS [36] employs a neural machine translation model to automatically generate meaningful assertion statements for test methods, while ATHENATEST [33] uses a transformer-based model trained on developer-written test cases. These approaches produce more readable and accurate test cases, addressing key limitations of search-based methods.

With the rapid evolution of Large Language Models (LLMs), their ability to generate code has significantly improved, leading to their widespread adoption for automated test generation related tasks. Meta's TestGen-LLM [1] employs LLMs to automatically enhance existing human-written tests. LIBRO [14] generates test cases directly from bug reports. Cedar [22] introduces an automated retrieval-based demonstration selection method to improve test assertion generation. CodaMosa[15] enhances the search-based test generation tool Pynguin[17, 18] by incorporating LLMs to overcome coverage plateaus. UGen [7] integrates search-based software testing with LLMs to improve the understandability of unit tests generated by EvoSuite. ChatUniTest [4] follows a generation-validation-repair mechanism to refine LLM-generated tests. ASTER [24] describes a generic pipeline applicable to multiple programming languages and complex software systems that require environment mocking. Empirical studies have also been conducted to evaluate the test generation capabilities of different LLMs across various programming languages, including JavaScript [31], Java [32, 38], and Python [34].

However, the empirical studies [8, 31, 32, 34] highlight a persistent challenge: LLMs struggle to generate unit tests for complex methods and often fail to achieve sufficient branch coverage, particularly in real-world projects. To address this limitation, recent research has explored strategies to provide LLMs with additional program-related information beyond just the focal class or focal method, enhancing their ability to generate tests for complex code.

HITS [35] prompts LLMs to decompose focal methods into slices and generate tests for each slice. However, these "slices" are not

derived from traditional program slicing techniques [37] but instead represent arbitrary method fragments generated by LLMs.

SymPrompt [28] employs symbolic execution to extract path constraints from each method under test. It then reduces the extensive set of extracted paths to a manageable subset of constraints and generates tests incrementally, path by path. In contrast, PANTA leverages a path approximation algorithm that directly computes a linearly independent set of execution paths from the control flow graph. Moreover, while SymPrompt provides only branch conditions, PANTA delivers full execution paths, enabling the LLM to more accurately infer the correct test inputs.

Additionally, both SymPrompt and HITS provide static prompts containing method fragments (slices or paths) but lack guidance on where additional tests are needed, restricting them to generating new test classes rather than augmenting existing ones. CoverUp [26], on the other hand, provides code coverage information to indicate which parts of the code remain uncovered but does not guide the LLM on how to cover those uncovered segments effectively.

Unlike existing techniques, PANTA adopts an iterative workflow that leverages hybrid program analysis to guide the LLM. It integrates dynamic code coverage analysis and static control flow analysis to systematically extract and rank uncovered paths. These paths then help the LLM in reasoning about where and how to generate tests. Also, unlike previous methods, our approach effectively supports both augmenting existing tests and generating new test classes when no tests exist for a class/method.

8 Conclusion

This paper introduced PANTA, a novel technique for automated unit test generation that seeks to emulate how human developers create tests. It addresses the challenge of achieving high branch coverage in complex code, where large language models often fall short. PANTA employs an iterative approach, combining static control flow analysis to identify execution paths with dynamic code coverage analysis to pinpoint under-tested areas. This hybrid program analysis guides an LLM to generate targeted test cases, which are then validated and repaired, using feedback to refine subsequent test generation efforts. Our evaluations on complex classes from the Defects4J [13] benchmark show PANTA significantly outperforms existing methods in line and branch coverage, achieving 26% higher line coverage and 23% higher branch coverage compared to the state-of-the-art, demonstrating the effectiveness of its iterative, program analysis-guided strategy for improving test thoroughness.

Looking ahead, we plan to extend PANTA to support multiple programming languages, broadening its applicability across diverse software projects. Additionally, we aim to incorporate cross-class context to further reduce hallucination and improve the accuracy of generated test cases.

References

- [1] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated unit test improvement using large language models at meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 185–196.
- [2] Anthropic. 2025. Claude 3.5 Haiku. <https://www.anthropic.com/claude/haiku>. Accessed: March 4, 2025.
- [3] atlassian. 2025. What is code coverage? <https://www.atlassian.com/continuous-delivery/software-testing/code-coverage>. Accessed: March 4, 2025.
- [4] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 572–576.
- [5] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. Pit: a practical mutation testing tool for java. In *Proceedings of the 25th international symposium on software testing and analysis*. 449–452.
- [6] Debeshee Das, Noble Saji Mathews, Alex Mathai, Srikanth Tamilselvan, Kranthi Sedamaki, Sridhar Chimalakonda, and Atul Kumar. 2023. COMEX: A Tool for Generating Customized Source Code Representations. In *38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2054–2057.
- [7] Amirhossein Deljouyi, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. 2024. Leveraging Large Language Models for Enhancing the Understandability of Generated Unit Tests. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 392–404.
- [8] Khalid El Haji. 2023. *Empirical study on test generation using GitHub Copilot*. Master's thesis. Delft University of Technology.
- [9] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic Test Suite Generation for Object-Oriented Software. In *19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE '11)*. ACM, 416–419.
- [10] Gordon Fraser and Andrea Arcuri. 2013. Evosuite: On the challenges of test case generation in the real world. In *2013 IEEE sixth international conference on software testing, verification and validation*. IEEE, 362–369.
- [11] geeksforgeeks. 2025. JaCoCo Java Code Coverage Library. <https://www.jacoco.org/jacoco/>. Accessed: March 4, 2025.
- [12] geeksforgeeks. 2025. What is Branch Coverage in Unit Testing? <https://www.geeksforgeeks.org/what-is-branch-coverage-in-unit-testing/>. Accessed: March 4, 2025.
- [13] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.
- [14] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large Language Models are Few-Shot Testers: Exploring LLM-Based General Bug Reproduction. In *Proceedings of the International Conference on Software Engineering (Melbourne, Victoria, Australia) (ICSE'23)*. IEEE Press, 2312–2323. doi:10.1109/ICSE48619.2023.00194
- [15] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-Trained Large Language Models. In *45th International Conference on Software Engineering (ICSE '23)*. IEEE Press, 919–931.
- [16] Llama. 2025. The open-source AI models you can fine-tune, distill and deploy anywhere. <https://www.llama.com/>. Accessed: March 4, 2025.
- [17] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: Automated Unit Test Generation for Python. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 168–172. doi:10.1145/3510454.3516829
- [18] Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2020. Automated unit test generation for python. In *Search-Based Software Engineering: 12th International Symposium, SSBSE 2020, Bari, Italy, October 7–8, 2020, Proceedings 12*. Springer, 9–24.
- [19] Leonard A Marascuilo and Ronald C Serlin. 1988. *Statistical methods for the social and behavioral sciences*. WH Freeman/Times Books/Henry Holt & Co.
- [20] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. doi:10.1109/TSE.1976.233837
- [21] Mistral. 2025. Mistral Large 2. <https://mistral.ai/news/mistral-large-2407>. Accessed: March 4, 2025.
- [22] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning. In *45th International Conference on Software Engineering (ICSE '23)*. IEEE Press, 2450–2462.
- [23] OpenAI. 2025. GPT-4o mini: advancing cost-efficient intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>. Accessed: March 4, 2025.
- [24] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. 2025. ASTER: Natural and Multi-language Unit Test Generation with LLMs. arXiv:2409.03093 [cs.SE] <https://arxiv.org/abs/2409.03093>
- [25] Panta. 2025. LLM Test Generation via Iterative Hybrid Program Analysis. <https://github.com/PANTA-TestAutomation/Panta>. Accessed: August 20, 2025.
- [26] Juan Altmayer Pizzorno and Emery D Berger. 2024. Coverup: Coverage-guided llm-based test generation. *arXiv preprint arXiv:2403.16218* (2024).
- [27] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J Hellendoorn. 2023. CAT-LM training language models on aligned code and tests. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 409–420.

- [28] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM. *Proc. ACM Softw. Eng.* FSE, Article 43 (July 2024), 21 pages. doi:10.1145/3643769
- [29] salesforce. 2025. Code Coverage Best Practices. https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_code_coverage_best_pract.htm. Accessed: March 4, 2025.
- [30] BA Sanusi, SO Olabiyisi, AO Afolabi, AO Olowoye, et al. 2020. Development of an Enhanced Automated Software Complexity Measurement System. *Journal of Advances in Computational Intelligence Theory* 1, 3 (2020), 1–11.
- [31] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2023), 85–105.
- [32] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2024. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 313–322.
- [33] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020).
- [34] Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2024. Testeval: Benchmarking large language models for test case generation. *arXiv preprint arXiv:2406.04531* (2024).
- [35] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1258–1268. <https://doi.ieeecomputersociety.org/>
- [36] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (Seoul, South Korea) (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 1398–1409. doi:10.1145/3377811.3380429
- [37] Mark Weiser. 1984. Program slicing. *IEEE Transactions on software engineering* SE-10, 4 (1984), 352–357.
- [38] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, et al. 2024. On the evaluation of large language models in unit test generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1607–1619.
- [39] Yucheng Zhang and Ali Mesbah. 2015. Assertions are strongly correlated with test suite effectiveness. In *Proceedings of the Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 214–224. doi:10.1145/2786805.2786858